Vietnam National University Ho Chi Minh City
International University
School of Computer Science and Engineering

THESIS - SEMESTER 2 (2025 - 2026)

Full name: Ngo Thi Thuong

Student ID: ITCSIU21160

THESIS TITLE: A Deep Learning Approach for Artwork Reconstruction

(An extended framework based on Mask-Aware Transformer - MAT)

GitHub Link: thuongngo050902/THESIS

GitHub Link: thuongngo050902/THESIS

This document provides a detailed guide on setting up the environment and running experiments for the portrait artwork restoration system (FaceArt Restoration) using our proposed deep learning model based on the Mask-Aware Transformer (MAT) architecture. The guide covers the project structure, environment configuration for NVIDIA GPUs, Apple Silicon MacBooks (MPS), running the interactive Streamlit Web UI, using the CLI for inference, and the three-stage model retraining workflow to reproduce the results.

I. System Overview & Repository Structure

1. Introduction

This system restores damaged portrait artwork (scratches, cracks, missing structural details) using our proposed deep learning model based on the Mask-Aware Transformer (MAT) architecture. Unlike general-purpose image restoration methods, this model is specifically optimized for artistic portrait faces using Relative Position Bias, Transformer Adapters, and Structure Guidance to preserve brushstrokes and facial geometric alignment.

2. Directory Structure

Below is the core structure of the repository outlining the function of key folders and scripts:

- [checkpoints/](file:///d:/2025-2026/Thesis/Clone/MAT/checkpoints) — Directory storing trained model weights (`.pkl` snapshot files).
- [collab/](file:///d:/2025-2026/Thesis/Clone/MAT/collab) — Scripts and utilities for training and inference on Colab or remote GPU servers.
- [run_phase1_server_from_drive.sh](file:///d:/2025-2026/Thesis/Clone/MAT/collab/run_phase1_server_from_drive.sh) — Automatic script to download data and train Phase 1.
- [run_phase3_server_from_phase2.sh](file:///d:/2025-2026/Thesis/Clone/MAT/collab/run_phase3_server_from_phase2.sh) — Automatic script to resume Phase 2 and train Phase 3.
- [datasets/](file:///d:/2025-2026/Thesis/Clone/MAT/datasets) — Data pipeline implementation and degradation mask generation.
- [dataset_512.py](file:///d:/2025-2026/Thesis/Clone/MAT/datasets/dataset_512.py) — Training data loader for 512x512 resolution.
- [mask_generator_512.py](file:///d:/2025-2026/Thesis/Clone/MAT/datasets/mask_generator_512.py) — Script simulating irregular paint-loss damage masks.
- [demo_ui/](file:///d:/2025-2026/Thesis/Clone/MAT/demo_ui) — Support scripts for the Streamlit web application.
- [inference_adapter.py](file:///d:/2025-2026/Thesis/Clone/MAT/demo_ui/inference_adapter.py) — Adapter interface routing inference to local device or FastAPI backend.
- [parf_compare.py](file:///d:/2025-2026/Thesis/Clone/MAT/demo_ui/parf_compare.py) — Layout logic for side-by-side comparative visualization.
- `dnnlib/` — Lower-level utility libraries for configuration and logging (adapted from StyleGAN2-ADA).

- [evaluation/](file:///d:/2025-2026/Thesis/Clone/MAT/evaluation) — Scripts for quantitative evaluation of restoration quality.
- `eval_psnr_ssim.py` — Measures Peak Signal-to-Noise Ratio (PSNR) and Structural Similarity (SSIM).
- `eval_fid.py` — Measures Fréchet Inception Distance (FID) to evaluate texture realism.
- [losses/](file:///d:/2025-2026/Thesis/Clone/MAT/losses) — Definition of loss functions used during optimization.
- [loss.py](file:///d:/2025-2026/Thesis/Clone/MAT/losses/loss.py) — Core two-stage loss of the Mask-Aware Transformer.
- [focal_frequency_loss.py](file:///d:/2025-2026/Thesis/Clone/MAT/losses/focal_frequency_loss.py) — Focal Frequency Loss (FFL) to improve paint texture details.
- [networks/](file:///d:/2025-2026/Thesis/Clone/MAT/networks) — Deep neural network architecture files.
- [mat.py](file:///d:/2025-2026/Thesis/Clone/MAT/networks/mat.py) — Main Mask-Aware Transformer model with window-based attention layers.
- [structure_guidance.py](file:///d:/2025-2026/Thesis/Clone/MAT/networks/structure_guidance.py) — Geometric structure guidance network used in Phase 3.
- `torch_utils/` — Helper scripts interfacing directly with PyTorch.
- `training/` — Core training loops and hyperparameter optimization scripts.
- [dataset_tool.py](file:///d:/2025-2026/Thesis/Clone/MAT/dataset_tool.py) — Preprocessing script packing raw images into uncompressed ZIP datasets for fast disk I/O.
- [generate_image.py](file:///d:/2025-2026/Thesis/Clone/MAT/generate_image.py) — CLI tool for fast batch inference on images.
- [requirements.txt](file:///d:/2025-2026/Thesis/Clone/MAT/requirements.txt) — Full list of python library dependencies.
- [streamlit_app.py](file:///d:/2025-2026/Thesis/Clone/MAT/streamlit_app.py) — Main entrypoint starting the interactive Streamlit Web UI.

II. System Requirements & Environment Setup

1. Hardware Requirements

- **For Inference & Web UI Demo:**
- **OS:** Windows 10/11, macOS Big Sur or newer (Intel & Apple Silicon M1/M2/M3 supported), or Linux Ubuntu 18.04+.
- **RAM:** Minimum 16 GB.
- **GPU Acceleration:**
- **Windows/Linux:** CUDA-capable NVIDIA GPU (Minimum 6-8 GB VRAM recommended).
- **MacBook:** Apple Silicon M-series (using Metal Performance Shaders - MPS on the integrated GPU).
- **CPU-only:** Supported on all platforms (slower processing speed).
- **For Model Training:**
- **OS:** Linux Ubuntu 20.04/22.04 LTS (Recommended).
- **GPU:** Dedicated NVIDIA GPUs (RTX 3090, RTX 4090, A100, RTX A6000) with >= 16GB-24GB VRAM.
- **CPU:** Intel Xeon or AMD EPYC (8+ cores).

2. Virtual Environment Setup

We recommend using Conda or Miniconda to manage Python packages and prevent C++ library compiler conflicts:

Step 1: Create a Python 3.8 virtual environment

```
conda create -n faceart_env python=3.8 -y
conda activate faceart_env
```

- **Alternative (Using traditional Python venv):**
- *On Windows (PowerShell):*

```
    python -m venv .venv
    .venv\Scripts\Activate.ps1
```

- *On Linux/macOS:*

```
    python3 -m venv .venv
    source .venv/bin/activate
```

Step 2: Install PyTorch

Choose the command matching your hardware setup:

- **For NVIDIA GPUs (CUDA 11.8 - Recommended):**

```
  pip install torch==2.1.2 torchvision==0.16.2 --index-url
https://download.pytorch.org/whl/cu118
```

- **For Apple Silicon MacBooks (MPS - M1/M2/M3):**

```
  pip install torch torchvision
```

- **For CPU-only:**

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
```

Step 3: Install dependencies and Ninja

Install the python libraries listed in `requirements.txt`:

```
pip install --upgrade pip
pip install -r requirements.txt
```

> ***Ninja Compiler:***
>
> *The `ninja` compiler is required to build optimized CUDA kernels. Install it using:*
>
> ** Conda: `conda install -c conda-forge ninja -y`*
>
> ** Pip: `pip install ninja`*
>
> *For CPU or macOS (MPS) environments, if custom CUDA operators fail to compile, the code falls back to CPU implementations automatically.*

III. Running the Streamlit Web UI

The repository provides an interactive Streamlit Web UI (`streamlit_app.py`) allowing users to upload damaged portrait images, draw masks, and run multi-stage restoration.

1. Launch the Streamlit application

Run the following command from the root workspace directory with the virtual environment activated:

```
streamlit run streamlit_app.py --server.port 8501 --server.address 127.0.0.1
```

Once running, open your web browser and navigate to: http://127.0.0.1:8501

2. Backend & Device Configuration (Advanced Controls)

Expand **Advanced Controls** in the left sidebar to configure inference settings:

- **Option A: Local Backend (Running models on your machine)**
- Best for personal computers with NVIDIA GPUs or Apple Silicon Macbooks.
- Download the trained model checkpoints (`.pkl`) and place them in the `checkpoints/` folder:
- Stage 1 Coarse: `resume_phase1_from_finetune_plus_loss.pkl`
- Final Output (Phase 3): `network-snapshot-000072.pkl`
- Original MAT (Baseline): `Places_512_FullData.pkl`
- Configure on UI:
- Set `Backend Mode` to `local`.
- Set `Device` to `cuda` (NVIDIA GPU), `mps` (Apple Silicon MacBook), or `cpu`.
- Enter the relative file paths of the checkpoints in their input fields.
- **Option B: Remote Backend (Running models on a remote GPU server or Colab)**
- Useful for lightweight laptops (e.g. standard MacBooks) to offload heavy neural computations to a GPU server over the network.
- On the remote GPU server, start the FastAPI API server:

```
uvicorn colab_inference_api:app --host 0.0.0.0 --port 8000
```

- On your local laptop (Client UI):
- Set `Backend Mode` to `remote`.
- Set `Remote endpoint` to the server API URL (e.g., `http://<server-ip>:8000` or a public ngrok tunnel address generated by Google Colab).

3. Streamlit UI Restoration Workflow

The web UI is divided into step-by-step stages mapping directly to the MAT restoration pipeline:

1. **Input & Mask:** Upload a damaged portrait image. Paint a mask over the cracks and damages using the freehand **Drawable Canvas**, or select position-based presets (Center, Left, Right, etc.) and click "Generate Mask". Use direction adjustment buttons (`←`, `→`, `↑`, `↓`) to nudge the mask position to cover the damage.
2. **Masked Input:** Preview the input package showing the original image overlayed with a dark mask. Click **Confirm Input Package** to lock configuration.

3. **Stage 1 (Coarse Restoration):** Uses the model trained in Phase 1 (Base Finetuning + Reconstruction Loss) to restore a coarse layout, filling in large structure holes.
4. **Final Output (Refinement Stage):** Uses the model trained in Phase 2 & 3 (incorporating Transformer Adapters, Structure Guidance, and Adaptive Gate) to restore fine details (eyes, nose, mouth lines, and brushstroke textures).
5. **Compare View & Lightbox:** Select the **Compare** tab to compare the original image side-by-side with the Stage 1 coarse output, Final Output, and original MAT Baseline model. Toggle **Lightbox** mode to zoom (mouse scroll) and pan (drag and drop) to inspect fine brushstroke transitions.

IV. Command-Line Interface (CLI) Batch Inference

To perform batch inference on a directory of images without loading the graphical interface, run the `generate_image.py` script:

```
python generate_image.py \
    --network checkpoints/network-snapshot-000072.pkl \
    --dpath test_sets/CelebA-HQ/images \
    --mpath test_sets/CelebA-HQ/masks \
    --outdir samples_output
```

Parameter Reference:

- `--network`: File path to the trained checkpoint snapshot (`.pkl`).
- `--dpath`: Directory containing the input damaged portrait images.
- `--mpath`: *(Optional)* Directory containing binary black-and-white mask images (0/black for valid context, 1/white for corrupted holes to restore). If omitted, random masks will be generated automatically.
- `--outdir`: Directory path where restored portrait images will be saved.

> ***Image Dimensions Constraint:***
>
> *The MAT model requires image width and height to be multiples of 512 (e.g., 512x512, 1024x1024). For arbitrary dimensions, resize or pad the image to a multiple of 512 before running the script. The Streamlit Web UI handles this padding automatically.*

V. Model Retraining Workflow

The retraining process of the artwork restoration model is divided into three consecutive training stages:

1. Dataset Preparation

Compress the training dataset into an uncompressed ZIP format to maximize GPU disk read throughput (I/O) during training:

```
python dataset_tool.py --source /path/to/raw/images --dest datasets/faceart_train.zip --width
512 --height 512
```

Prepare both `datasets/faceart_train.zip` (training set) and `datasets/faceart_val.zip` (validation set).

2. Three-Phase Training Flow

Phase 1: Base Finetuning

- **Objective:** Finetune the base MAT network on the custom portrait artwork dataset (FaceArt), enabling Relative Position Bias (to model brushstroke spatial relationships), Mask-Aware Additive Bias (to suppress halo artifacts), and Deterministic Latent Gate (to stabilize style mixing).
- **Automation scripts on server:** `bash collab/run_phase1_server_from_drive.sh prepare`, then run `train` inside a tmux session.
- **Core training command:**

```
  python train.py \
      --outdir=runs/faceart_phase1_relbias_gate \
      --data=datasets/faceart_train.zip \
      --data_val=datasets/faceart_val.zip \
      --cfg=places512 \
      --resume=checkpoints/Places_512_FullData.pkl \
      --epochs=40 --batch=4 --workers=2 --snap=5 --metrics=none \
      --pr=0.1 --pl=False --truncation=0.5 --style_mix=0.5 --ema=10 --lr=5e-5 --lrt=1e-4
--lambda-ffl=0.02 --aug=noaug \
      --enable-rel-pos-bias=True --enable-mask-bias=True
--enable-deterministic-latent-gate=True
```

Phase 2: Transformer Adapter Training

- **Objective:** Freeze the core parameters of MAT learned in Phase 1 and insert lightweight adaptable blocks (Transformer Adapters at resolutions of 32x32 and 16x16) between attention layers. This enables the model to adapt to portrait artwork style details without destroying its general restoration knowledge.
- **Core training command:**

```
  python train.py \
      --outdir=runs/faceart_phase2_tran_adapter \
      --data=datasets/faceart_train.zip \
      --data_val=datasets/faceart_val.zip \
      --cfg=places512 \
      --resume=runs/faceart_phase1_relbias_gate/00000-places512/network-snapshot-000040.pkl \
      --epochs=30 --batch=4 --workers=2 --snap=5 --metrics=none \
      --pr=0.1 --pl=False --truncation=0.5 --style_mix=0.5 --ema=10 --lr=2.5e-5 --lrt=1e-4
--lambda-ffl=0.02 --aug=noaug \
      --enable-rel-pos-bias=True --enable-mask-bias=True
--enable-deterministic-latent-gate=True \
      --enable-tran-adapter-32=True --enable-tran-adapter-16=True
```

Phase 3: Structure Guidance & Adaptive Gate

- **Objective:** Activate the portrait structure guidance network (Structure Guidance) to restore large missing components, and train the adaptive gate module (Adaptive Gate) to merge global facial structure details with local adapter textures.
- **Automation scripts on server:** `bash collab/run_phase3_server_from_phase2.sh prepare`, then run `train` inside tmux.
- **Core training command:**

```
  python train.py \
      --outdir=runs/faceart_phase3_adaptive_structure_guidance \
      --data=datasets/faceart_train.zip \
      --data_val=datasets/faceart_val.zip \
      --cfg=places512 \
      --resume=runs/faceart_phase2_tran_adapter/00000-places512/network-snapshot-000030.pkl \
      --epochs=10 --batch=4 --workers=2 --snap=2 --metrics=none \
      --pr=0.1 --pl=False --truncation=0.5 --style_mix=0.5 --ema=10 --lr=5e-5 --lrt=1e-4
--lambda-ffl=0.02 --aug=noaug \
      --enable-rel-pos-bias=True --enable-mask-bias=True
--enable-deterministic-latent-gate=True \
      --enable-tran-adapter-32=True --enable-tran-adapter-16=True \
      --enable-structure-guidance=True --enable-structure-fuse-16=True
--enable-structure-fuse-stage2=True --enable-structure-fuse-32=False
--enable-adaptive-structure-gate=True
```

3. Training Monitoring

To monitor training loss values and check progress:

- Read the execution logs in your current training directory:

```
  tail -f runs/faceart_phase3_adaptive_structure_guidance/00000-places512/log.txt
```

VI. Model Evaluation Metrics

The evaluation scripts are stored in the `evaluation/` directory, allowing you to calculate quantitative image quality metrics:

1. **PSNR, SSIM, and L1 Loss:**

```
   python evaluation/eval_psnr_ssim.py --rec_path /path/to/reconstructed/images --gt_path
/path/to/ground_truth/images
```

2. **LPIPS (Human Perceptual Quality Metric):**

```
   python evaluation/eval_lpips.py --rec_path /path/to/reconstructed/images --gt_path
/path/to/ground_truth/images
```

3. **FID (Fréchet Inception Distance for image realism):**

```
   python evaluation/eval_fid.py --rec_path /path/to/reconstructed/images --gt_path
/path/to/ground_truth/images
```